ReconForge Usage Guide

> Version 1.0 — Last updated: 2026-03-21

CLI Entry Point

```
python reconforge.py <module> [options]
```

Available modules: `network`, `ad`, `web`, `api`, `surface`, `workflow`

Common Options

These flags are available across all modules unless noted otherwise:

Flag	Default	Description
`-t, --target`	(required)	Target specification (IP, CIDR, hostname, or URL)
`--opsec`	`normal`	OPSEC mode: `stealth`, `normal`, `aggressive`
`--phases`	all	Comma-separated phase list
`-o, --output`	`outputs`	Output base directory
`-v, --verbose`	off	Verbose/debug output
`--dry-run`	off	Show commands without executing
`--timeout`	600	Default command timeout in seconds
`--encrypt-loot`	off	Encrypt loot files with Fernet (`network`, `ad`, `web`, `api`, `workflow` only — **not available on the `surface` CLI**)

> **Note:** The `surface` subcommand does not expose `--encrypt-loot` on the CLI. The `SurfaceModule` Python class accepts `encrypt_loot` as a constructor parameter (defaulting to `False`), so programmatic callers and the workflow orchestrator can still enable it. Only the CLI flag is absent.

Network Module

```
# Basic scan
python reconforge.py network --target 10.10.10.1

# CIDR range in stealth mode
python reconforge.py network --target 10.10.10.0/24 --opsec stealth

# Aggressive with brute-force (opt-in)
python reconforge.py network --target 10.10.10.1 --opsec aggressive --brute-force

# Specific phases only
python reconforge.py network --target 10.10.10.1 --phases discovery,scanning -v
```

Phases

Phase	Slug	Description
Host Discovery	`discovery`	Ping sweep, ARP scan, live host identification
Port Scanning	`scanning`	SYN/connect scan, service detection
Service Enumeration	`enumeration`	SMB, LDAP, deep service analysis
Authentication Checks	`authentication`	Anonymous access, credential testing

Network-Specific Flags

Flag	Description
`--brute-force`	Enable hydra brute-force testing (opt-in only)

AD Module

```
# Basic AD recon
python reconforge.py ad --target 10.10.10.1 --domain corp.local

# Stealth mode (passive only)
python reconforge.py ad --target 10.10.10.1 --domain corp.local --opsec stealth

# Authenticated enumeration
python reconforge.py ad --target 10.10.10.1 --domain corp.local -u admin -p
'P@ssw0rd' --dc-ip 10.10.10.1

# Specific phases
python reconforge.py ad --target 10.10.10.1 --domain corp.local --phases pass-
ive,identity -v
```

Phases

Phase	Slug	Description
Passive Recon	`passive`	DNS SRV, anonymous LDAP, SMB null sessions
Identity Enumeration	`identity`	Users, groups, RID cycling, Kerberos enumeration
Configuration Enumeration	`configuration`	GPO, trusts, misconfiguration analysis
Delegation Discovery	`delegation`	Unconstrained/constrained/ RBCD delegation
BloodHound Collection	`bloodhound`	AD graph data collection

AD-Specific Flags

Flag	Description
`--domain`	AD domain name (e.g., `corp.local`)
`-u, --username`	Username for authenticated enumeration
`-p, --password`	Password for authenticated enumeration
`--dc-ip`	Domain Controller IP (if different from target)

Web Module

```
# Basic web recon
python reconforge.py web --target https://example.com

# Stealth mode
python reconforge.py web --target https://example.com --opsec stealth

# Full aggressive scan including exploit phase
python reconforge.py web --target https://example.com --opsec aggressive --phases sur-
face,content,vuln,exploit

# With custom extensions
python reconforge.py web --target https://example.com --phases surface,content -e php,
asp -v
```

Phases

Phase	Slug	Description
Surface Discovery	`surface`	Technology fingerprinting, WAF detection
Content Enumeration	`content`	Directory brute-forcing, file discovery
Vulnerability Scanning	`vuln`	Nuclei templates, nikto, CVE checks
Exploit Candidates	`exploit`	SQLMap, exploit identification (opt-in)

Web-Specific Flags

Flag	Description
`-w, --wordlist`	Custom wordlist for content discovery
`-th, --threads`	Thread count for fuzzing (default: 40)
`-e, --extensions`	File extensions to search (e.g., `php,asp,aspx`)
`--follow-redirects`	Follow HTTP redirects (default: true)
`--verify-ssl`	Verify SSL certificates

API Module

```
# Basic API recon
python reconforge.py api --target https://api.example.com/v1

# Stealth mode (discovery only)
python reconforge.py api --target https://api.example.com --opsec stealth

# With authentication token
python reconforge.py api --target https://api.example.com --auth-token "Bearer eyJ..."

# Full scan including authorization testing
python reconforge.py api --target https://api.example.com --phases discov-
ery,authentication,fuzzing,authorization -v
```

Phases

Phase	Slug	Description
Discovery	`discovery`	OpenAPI spec detection, endpoint enumeration
Authentication	`authentication`	JWT analysis, token testing, auth scheme detection
Fuzzing	`fuzzing`	Parameter fuzzing, endpoint fuzzing
Authorization	`authorization`	IDOR testing, privilege escalation checks (opt-in)

API-Specific Flags

Flag	Description
`--auth-token`	Bearer/auth token for authenticated requests
`--header`	Extra HTTP header (repeatable, e.g., `--header 'X-Api-Key: abc'`)
`-w, --wordlist`	Custom wordlist for endpoint discovery

Surface Module

```
# Basic attack surface mapping
python reconforge.py surface --target 10.10.10.1

# Stealth mode
python reconforge.py surface --target 10.10.10.1 --opsec stealth

# Aggressive full-port scan
python reconforge.py surface --target 10.10.10.1 --opsec aggressive
```

Phases

Phase	Slug	Description
Port Discovery	`port_discovery`	OPSEC-aware port scanning
Service Fingerprint	`service_fingerprint`	Version detection, banner grabbing
Vector Correlation	`vector_correlation`	Correlates ports/services/URLs into attack surface map
Prioritization	`prioritization`	Ranks attack vectors by risk and confidence

---

Workflow Orchestrator

```
# Full recon (conditional branching)
python reconforge.py workflow --target 10.10.10.1

# Targeted modules
python reconforge.py workflow --target 10.10.10.1 --modules network,ad,web

# With engagement tracking
python reconforge.py workflow --target 10.10.10.1 --engagement "Q1 Pentest" --client "
Acme Corp" --operator "Andrews"

# Stealth full recon
python reconforge.py workflow --target 10.10.10.1 --opsec stealth

# Resume a paused engagement
python reconforge.py workflow --target 10.10.10.1 --resume /path/to/engagement.json
```

Workflow-Specific Flags

Flag	Description
`--modules`	Comma-separated modules to run (default: full-recon pipeline)
`--engagement`	Engagement name
`--client`	Client name for engagement tracking
`--operator`	Operator name (default: “Andrews Ferreira”)
`--resume`	Path to saved engagement JSON to resume

Full Recon Pipeline

When no `--modules` flag is given, the workflow runs:

```
surface → network → ad (if AD services detected) → web (if HTTP detected) → api (if HTTP detected)
```

OPSEC Modes

Mode	Noise Levels	Behavior
`stealth`	low only	Minimal noise, avoids detection. Limited port range, no version detection, no scripts, passive techniques preferred
`normal`	low, medium	Balanced for standard engagements. Full port range, version detection enabled, moderate enumeration
`aggressive`	low, medium, high, very_high	Maximum coverage for CTF/lab environments. All ports, all scripts, full enumeration, UDP scanning

Techniques exceeding the mode’s allowed noise levels are **blocked** with a logged warning.

Output Structure

All output is written under `<output_base>/<target>/`:

```
outputs/10.10.10.1/
├── network/
│   ├── raw/                # Raw tool output files
│   ├── parsed/             # Parsed/structured results
│   ├── findings.json       # JSON findings
│   ├── findings.md         # Markdown findings
│   ├── loot.json           # Discovered loot
│   ├── session.md          # Session notes timeline
│   ├── commands.log        # All commands executed
│   ├── attack_paths.md     # Attack path documentation
│   └── quick_report.md     # Quick summary
├── ad/
│   └── ...                 # Same structure per module
├── web/
│   └── ...
├── api/
│   └── ...
├── surface/
│   └── ...
└── engagement_report.md    # Unified engagement report (workflow mode)
```

Loot Encryption

When `--encrypt-loot` is specified:
- Loot files are written as Fernet-encrypted blobs with `.enc` suffix
- Encryption key is stored at `~/.reconforge/loot.key` (mode 0600)
- Credential vault files are similarly encrypted

Examples

Quick Network Scan

```
python reconforge.py network -t 10.10.10.1 --phases discovery,scanning
```

Full AD Engagement

```
python reconforge.py workflow -t 10.10.10.1 \
    --modules network,ad \
    --opsec normal \
    --engagement "Corp AD Assessment" \
    --client "Acme Corp" \
    --encrypt-loot
```

Stealth Web Assessment

```
python reconforge.py web -t https://target.com --opsec stealth -v
```

API Security Audit with Token

```
python reconforge.py api -t https://api.target.com/v2 \
    --auth-token "Bearer eyJhbGciOiJIUzI1NiIs..." \
    --phases discovery,authentication,fuzzing,authorization \
    --opsec aggressive -v
```

Usage guide validated: 2026-03-21 — 348/348 tests passing